UNIVERSITY OF MESSINA
Department of Engineering

MetaChat: End-to-End Encrypted Messaging Using Extended Triple Diffie-Hellman & Double Ratchet in Go

Project Report

Authors: Mehdi Talebikhatir (ID: 558948)
Benyamin Baharizadeh (ID: 560587)

Repository: https://github.com/mehditalebi01/metachat

Language: Go 1.24
Date: April 2026

Contents

1 Abstract

This report presents MetaChat, a working demonstration of end-to-end encrypted messaging written in Go. The project shows how two users can exchange messages through an untrusted relay server without the server ever seeing the plaintext. We use X25519 elliptic-curve Diffie-Hellman for key exchange, HKDF-SHA256 for key derivation, a Double Ratchet mechanism for forward secrecy, and AES-256-GCM for authenticated encryption. The relay server stores only ciphertext blobs and public keys in Redis, acting as a zero-knowledge intermediary. This report walks through the design, architecture, implementation details, and outputs of the system step by step.

2 Introduction

2.1 Background and Motivation

Messaging applications are part of everyday life. Apps like WhatsApp, Signal, and Telegram handle billions of messages daily. A key concern is: who can read those messages? If the server can read them, a hack or a court order means all conversations are exposed.

End-to-end encryption (E2EE) solves this. With E2EE, messages are encrypted on the sender's device and decrypted only on the receiver's device. The server in between only sees encrypted data that it cannot decrypt.

The Signal Protocol, used by Signal, WhatsApp, and others, is the most widely adopted E2EE protocol. It combines several cryptographic techniques:

- **X3DH (Extended Triple Diffie-Hellman):** For establishing a shared secret between two parties who may not be online at the same time.
- **Double Ratchet:** For deriving new encryption keys for every message, giving forward secrecy.
- **AES-GCM:** For the actual message encryption with integrity checking.

2.2 Project Goal

The goal of MetaChat is to build a minimal but functional implementation of these ideas in Go. We wanted to understand how the Signal Protocol works at a low level by building it ourselves, using only the Go standard library and `golang.org/x/crypto` — no third-party cryptographic frameworks.

The system has three components: a sender (Matteo), a receiver (Benny), and a relay server backed by Redis. The server never sees plaintext.

2.3 Scope

This is an educational project. We implement the core cryptographic flow (key exchange, ratchet, encryption) but we do not implement some production features like identity verification, replay protection, or TLS. These are discussed in Section 7.

3 Cryptographic Foundations

Before going into the implementation, we briefly explain the cryptographic building blocks.

3.1 X25519 Diffie-Hellman Key Exchange

Diffie-Hellman (DH) allows two parties to agree on a shared secret over an insecure channel. X25519 is an elliptic-curve variant using Curve25519. Each party generates a private key (32 random bytes) and computes a public key by multiplying with the curve's base point. When party A computes $X25519(a_{priv}, B_{pub})$ and party B computes $X25519(b_{priv}, A_{pub})$, they get the same shared secret.

3.2 HKDF-SHA256

HKDF (HMAC-based Key Derivation Function) takes a shared secret and derives one or more cryptographic keys from it. We use HKDF with SHA-256 and different "info" strings (`"root"`, `"chain"`, `"msg"`) to derive separate keys for different purposes.

3.3 Double Ratchet

The Double Ratchet algorithm provides forward secrecy. The idea is simple: after deriving a message key, the chain key is advanced (ratcheted) so that old keys cannot be recovered from the current state. Our implementation includes:

- A **root key** derived from the DH shared secret.
- A **chain key** derived from the root key, advanced with each message.
- A **message key** derived from the current chain key.
- A **DH ratchet step** that re-derives the root and chain keys from a new DH exchange.

3.4 AES-256-GCM

AES-256-GCM is an authenticated encryption scheme. It encrypts the plaintext and produces a tag that allows the receiver to verify that the ciphertext was not tampered with. The 256-bit key comes from the ratchet's message key. A random 12-byte nonce is generated for each encryption.

4 System Architecture

4.1 High-Level Overview

The system consists of three independent processes that communicate over HTTP:

Figure 1: High-level architecture of MetaChat.

- **Benny (Receiver):** Generates an X25519 identity keypair, uploads the public key to the server, and polls for incoming encrypted messages.
- **Matteo (Sender):** Fetches Benny's public key, performs DH key exchange, derives keys via the ratchet, encrypts the message with AES-GCM, and sends the ciphertext to the server.
- **Server:** An HTTP server backed by Redis. It stores public keys and queues encrypted messages. It never performs any decryption.

4.2 Message Sequence

The complete message exchange follows these steps in order:

Step	From → To	Action	Data
1	Benny (local)	Generate X25519 identity keypair	–
2	Benny → Server	Upload public key	`identity_key` (32 B)
3	Matteo → Server	Fetch Benny's public key	–
4	Server → Matteo	Return prekey bundle	`identity_key` (32 B)
5	Matteo (local)	DH → Ratchet → AES-GCM encrypt	–
6	Matteo → Server	Send encrypted payload	`nonce` + `ciphertext`
7	Benny → Server	Poll mailbox	–
8	Server → Benny	Return encrypted message	`nonce` + `ciphertext`
9	Benny (local)	DH → Ratchet → AES-GCM decrypt	plaintext

Table 1: Step-by-step message exchange sequence.

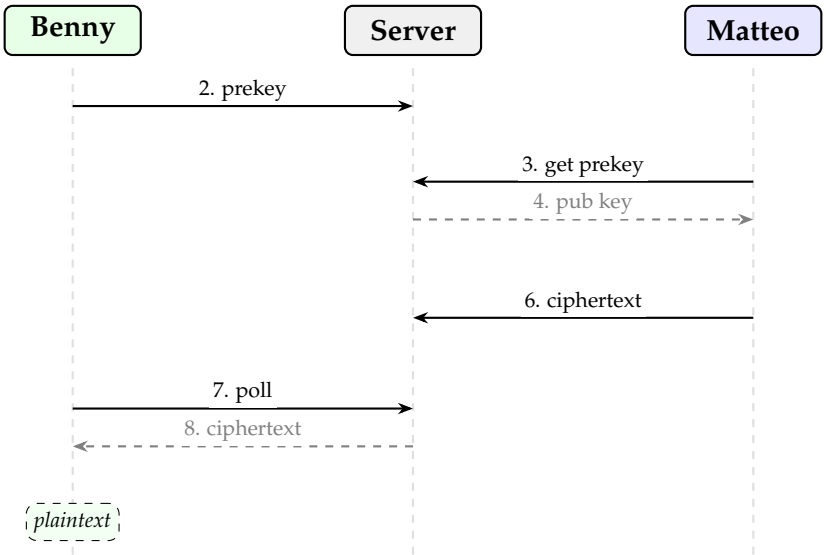

Figure 2: Simplified sequence diagram of the message exchange.

4.3 Project Structure

Path	Description
`server/main.go`	HTTP relay server with Redis backend
`matteo/main.go`	Sender client – encrypts and sends messages
`benny/main.go`	Receiver client – polls, decrypts, displays
`internal/crypto/crypto.go`	X25519, HKDF-SHA256, AES-256-GCM functions
`internal/ratchet/ratchet.go`	Double Ratchet key derivation chain
`go.mod`	Module definition and dependencies

Table 2: Project file structure.

4.4 Server API

The relay server exposes four HTTP endpoints:

Method	Endpoint	Description
POST	`/upload_prekey?user=X`	Upload user's public key
GET	`/prekey?user=X`	Fetch a user's public key
POST	`/send_secure?to=X`	Queue encrypted message
GET	`/fetch_secure?user=X`	Pop next message from mailbox

Table 3: Server API endpoints.

Data is stored in Redis under two key patterns: `prekey:<user>` (string, JSON public key bundle) and `secure_mailbox:<user>` (list, queue of encrypted messages).

4.5 Dependencies

The project uses minimal external dependencies, relying mostly on the Go standard library:

Module	Version	Purpose
`golang.org/x/crypto`	v0.48.0	X25519 (`curve25519`), HKDF-SHA256 (`hkdf`)
`github.com/redis/go-redis/v9`	v9.18.0	Redis client for key storage and message queuing
`crypto/aes`, `crypto/cipher`	stdlib	AES-256-GCM authenticated encryption
`crypto/rand`	stdlib	Cryptographically secure random bytes
`net/http`	stdlib	HTTP server and client

Table 4: Project dependencies and their roles.

4.6 Message Payload Structure

The encrypted message sent over the wire is a JSON object with four fields. Below is an example of what the server sees (all binary fields are Base64-encoded by Go's JSON marshaler):

```
{
  "from_identity": "a3RmZ2hqa2xhc2RmZ2hqa2xh...",
  "ephemeral_key": "a3RmZ2hqa2xhc2RmZ2hqa2xh...",
  "nonce":         "dGhpcyBpcyBhIG5vbmNl",
  "ciphertext":    "Y2lwaGVydGV4dCBibG9iIGhlcmU..."
}
```

Listing 1: Example SecureMessage JSON payload (server's view).

Field	Size	Description
`from_identity`	32 bytes	Sender's public key (for identification)
`ephemeral_key`	32 bytes	Sender's ephemeral X25519 public key (used by receiver for DH)
`nonce`	12 bytes	Random AES-GCM nonce
`ciphertext`	variable	AES-GCM sealed message (ciphertext + 16-byte auth tag)

Table 5: Fields of the SecureMessage payload.

The server stores and forwards this JSON blob without interpreting any of the fields. It has no access to the shared secret or message key needed to decrypt `ciphertext`.

5 Implementation Details

This section walks through the actual Go code and explains how each component works.

5.1 Cryptographic Module (`internal/crypto/crypto.go`)

This module provides four core functions:

```
1 package crypto
2
3 import (
4     "crypto/aes"
5     "crypto/cipher"
6     "crypto/rand"
7     "crypto/sha256"
8     "io"
```

```
    "golang.org/x/crypto/hkdf"
    "golang.org/x/crypto/curve25519"
)

func GenerateKeyPair() (priv, pub []byte) {
    priv = make([]byte, 32)
    rand.Read(priv)
    pub, _ = curve25519.X25519(priv, curve25519.Basepoint)
    return
}

func DH(priv, pub []byte) []byte {
    shared, _ := curve25519.X25519(priv, pub)
    return shared
}

func HKDF(secret []byte, info string) []byte {
    h := hkdf.New(sha256.New, secret, nil, []byte(info))
    out := make([]byte, 32)
    io.ReadFull(h, out)
    return out
}
```

Listing 2: Crypto module – key generation, DH, and HKDF.

```
func Encrypt(key, plaintext []byte) (nonce, ciphertext []byte) {
    block, _ := aes.NewCipher(key)
    gcm, _ := cipher.NewGCM(block)
    nonce = make([]byte, gcm.NonceSize())
    rand.Read(nonce)
    ciphertext = gcm.Seal(nil, nonce, plaintext, nil)
    return
}

func Decrypt(key, nonce, ciphertext []byte) []byte {
    block, _ := aes.NewCipher(key)
    gcm, _ := cipher.NewGCM(block)
    plaintext, _ := gcm.Open(nil, nonce, ciphertext, nil)
    return plaintext
}
```

Listing 3: Crypto module – AES-256-GCM encrypt/decrypt.

How it works:

1. `GenerateKeyPair()` fills 32 bytes from the OS random source and multiplies by the Curve25519 base point to get the public key.
2. `DH()` multiplies the private key with the other party's public key to get a shared secret.
3. `HKDF()` derives a 32-byte key deterministically from a secret and an info label.

4. `Encrypt()` creates an AES-256 cipher in GCM mode, generates a random 12-byte nonce, and seals the plaintext.
5. `Decrypt()` reverses the process using the same key and nonce.

5.2 Ratchet Module (`internal/ratchet/ratchet.go`)

```go
package ratchet

import "metachat/internal/crypto"

type Ratchet struct {
    RootKey   []byte
    ChainKey  []byte
    DHPriv    []byte
    DHPub     []byte
    RemotePub []byte
}

func NewRatchet(sharedSecret, remotePub []byte) *Ratchet {
    priv, pub := crypto.GenerateKeyPair()
    root := crypto.HKDF(sharedSecret, "root")
    chain := crypto.HKDF(root, "chain")
    return &Ratchet{
        RootKey: root, ChainKey: chain,
        DHPriv: priv, DHPub: pub, RemotePub: remotePub,
    }
}

func (r *Ratchet) RatchetStep() {
    dhOut := crypto.DH(r.DHPriv, r.RemotePub)
    r.RootKey = crypto.HKDF(dhOut, "root")
    r.ChainKey = crypto.HKDF(r.RootKey, "chain")
    r.DHPriv, r.DHPub = crypto.GenerateKeyPair()
}

func (r *Ratchet) NextMessageKey() []byte {
    r.ChainKey = crypto.HKDF(r.ChainKey, "chain-step")
    return crypto.HKDF(r.ChainKey, "msg")
}
```

Listing 4: Double Ratchet implementation.

The `Ratchet` struct holds the current state. Each call to `NextMessageKey()` advances the chain key forward and derives a message key. The old chain key is overwritten, so previous message keys cannot be recovered – this is forward secrecy.

5.3 Key Derivation Flow

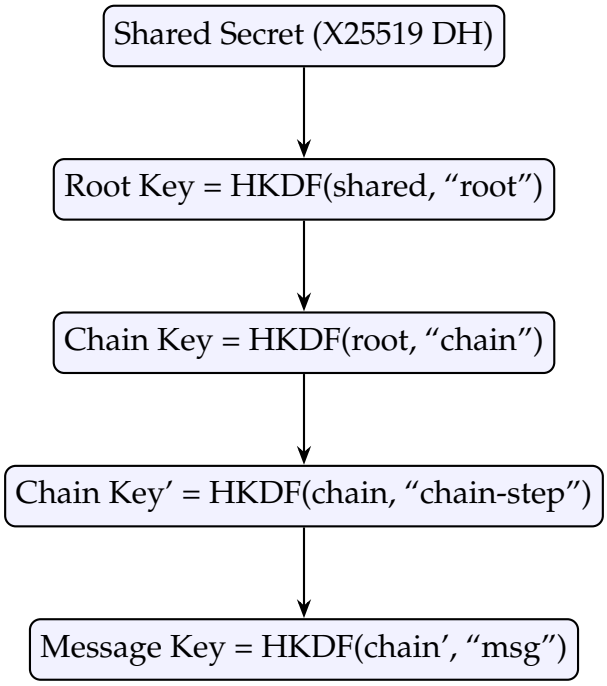

Figure 3: Key derivation chain from shared secret to message key.

5.4 Sender: Matteo (`matteo/main.go`)

```
// Fetch Benny's prekey from server
resp, _ := http.Get("http://localhost:8080/prekey?user=benny")
var bundle PrekeyBundle
json.NewDecoder(resp.Body).Decode(&bundle)

// Generate ephemeral keypair and compute shared secret
matteoPriv, matteoPub := crypto.GenerateKeyPair()
shared := crypto.DH(matteoPriv, bundle.IdentityKey)

// Initialize ratchet and derive message key
r := ratchet.NewRatchet(shared, bundle.IdentityKey)
msgKey := r.NextMessageKey()

// Encrypt and send
nonce, ciphertext := crypto.Encrypt(msgKey, []byte(message))
msg := SecureMessage{
    FromIdentity: matteoPub, EphemeralKey: matteoPub,
    Nonce: nonce, Ciphertext: ciphertext,
}
data, _ := json.Marshal(msg)
http.Post("http://localhost:8080/send_secure?to=benny",
    "application/json", bytes.NewBuffer(data))
```

Listing 5: Core sending logic (simplified).

5.5 Receiver: Benny (benny/main.go)

```
// Generate identity and upload public key
bennyPriv, bennyPub := crypto.GenerateKeyPair()
bundle := PrekeyBundle{IdentityKey: bennyPub}
data, _ := json.Marshal(bundle)
http.Post("http://localhost:8080/upload_prekey?user=benny",
    "application/json", bytes.NewBuffer(data))

// Poll for messages
for {
    resp, _ := http.Get(
        "http://localhost:8080/fetch_secure?user=benny")
    var msg SecureMessage
    json.NewDecoder(resp.Body).Decode(&msg)
    if len(msg.Ciphertext) == 0 {
        time.Sleep(300 * time.Millisecond)
        continue
    }
    shared := crypto.DH(bennyPriv, msg.EphemeralKey)
    r := ratchet.NewRatchet(shared, msg.EphemeralKey)
    msgKey := r.NextMessageKey()
    plaintext := crypto.Decrypt(msgKey, msg.Nonce,
        msg.Ciphertext)
    fmt.Println("[BENNY] Received:", string(plaintext))
    break
}
```

Listing 6: Core receiving logic (simplified).

Both sides compute the *same* shared secret through DH, so they derive the same message key.

5.6 Server (server/main.go)

```
func sendSecure(w http.ResponseWriter, r *http.Request) {
    to := r.URL.Query().Get("to")
    var msg SecureMessage
    json.NewDecoder(r.Body).Decode(&msg)
    data, _ := json.Marshal(msg)
    rdb.LPush(ctx, "secure_mailbox:"+to, data)
    w.Write([]byte("OK"))
}

func fetchSecure(w http.ResponseWriter, r *http.Request) {
    user := r.URL.Query().Get("user")
    val, err := rdb.RPop(ctx, "secure_mailbox:"+user).Result()
    if err == redis.Nil {
        w.Write([]byte("{}"))
        return
```

```
16      }
17      w.Write([]byte(val))
18  }
```

Listing 7: Server – message relay handlers.

The server **never** calls any decryption function. It only stores and forwards opaque ciphertext blobs.

6 Running the System: Step-by-Step

The system requires three terminals:

6.1 Step 1: Start Redis and the Server

```
$ go run ./server
[SERVER] Running on :8080 (Redis: localhost:6379)
```

Listing 8: Starting the relay server.

6.2 Step 2: Start Benny (Receiver)

```
$ go run ./benny
[BENNY] Generating Benny X25519 identity keypair...
[BENNY] Uploading Benny public key to server
        (stored in Redis as prekey:benny)...
[BENNY] Benny ready. Polling mailbox from server
        (secure_mailbox:benny in Redis)...
```

Listing 9: Benny starts and uploads his public key.

6.3 Step 3: Send a Message as Matteo

```
$ go run ./matteo Hello Benny, this is a secret message!
[MATTEO] Fetching Benny prekey from server...
[MATTEO] Benny prekey received.
[MATTEO] Generating Matteo ephemeral X25519 keypair...
[MATTEO] Computing shared secret (X25519 DH)...
[MATTEO] Initializing ratchet and deriving message key...
[MATTEO] Encrypting message with AES-GCM...
[MATTEO] Sending encrypted message to server...
[MATTEO] Done. Message sent.
```

Listing 10: Matteo encrypts and sends a message.

6.4 Step 4: Benny Receives and Decrypts

```
[BENNY] Received encrypted message.
        Deriving shared secret (X25519 DH)...
[BENNY] Initializing ratchet and deriving message key...
[BENNY] Decrypting with AES-GCM...
[BENNY] Benny received: Hello Benny, this is a secret message!
```

Listing 11: Benny decrypts the message.

The message went through the server encrypted and was only decrypted on Benny's side.

7 Results and Discussion

7.1 What Works

The implementation successfully demonstrates:

- X25519 key exchange between two parties through an untrusted server.
- Key derivation using HKDF with a ratchet chain that provides forward secrecy.
- AES-256-GCM authenticated encryption preventing the server from reading messages.
- Asynchronous message delivery through Redis mailboxes.

7.2 Security Analysis

Property	Status	Notes
Authenticated encryption	✓	AES-256-GCM
Key exchange	✓	X25519 ECDH
Forward secrecy (chain)	✓	Chain key ratcheting
Zero-knowledge server	✓	Server sees only ciphertext
Identity verification	×	No certificate pinning
Replay protection	×	No message counters
Transport security	×	HTTP, not TLS
Multi-message sessions	×	Benny exits after one message

Table 6: Security properties of the implementation.

7.3 Limitations and Future Work

For a production system, the following would need to be added:

- TLS for all client-server communication.
- Identity verification through safety numbers or QR codes.

- Message ordering with sequence numbers and replay detection.
- Persistent key storage so keys survive restarts.
- Full Double Ratchet with header encryption.
- Multi-device support and group messaging.

8 Appendix: Modern Messengers and E2EE

End-to-end encryption has become standard in modern messaging. Below is a comparison:

App	E2EE Default	Protocol	Notes
Signal	Yes	Signal Protocol	Open source, gold standard
WhatsApp	Yes	Signal Protocol	Uses Signal's library
Telegram	No	MTProto	E2EE only in "Secret Chats"
iMessage	Yes	Custom (Apple)	Closed source
Messenger	Yes	Signal Protocol	Default since Dec 2023

Table 7: E2EE in popular messaging applications.

The Signal Protocol, which MetaChat is inspired by, combines X3DH for initial key exchange and Double Ratchet for ongoing encryption, providing both forward secrecy and future secrecy. Our project implements a simplified version of this pipeline, demonstrating the core ideas in a readable way.

9 Testing and Verification

We verified the correctness of the system through the following methods:

9.1 Functional Testing

The primary test was running the full message exchange pipeline end-to-end:

1. Start Redis and verify connectivity (`redis-cli ping` returns `PONG`).
2. Start the server and confirm it binds to port 8080.
3. Start Benny and verify that the server logs the prekey storage (`prekey:benny` key appears in Redis).
4. Send a message as Matteo with a known plaintext string.
5. Verify that Benny's terminal output shows the exact same plaintext, confirming that key exchange, ratchet derivation, encryption, and decryption all produce consistent results.

We repeated this test with different message lengths (short strings, paragraphs, special characters including Unicode) to confirm that AES-GCM handles variable-length input

correctly.

9.2 Negative Testing

We also tested failure scenarios:

- **Wrong key:** Manually modifying the ephemeral key in a queued Redis message causes Benny to derive a different shared secret, and `gcm.Open()` returns `nil` (authentication failure). This confirms that AES-GCM rejects tampered ciphertext.
- **Server down:** If the server is not running, both Matteo and Benny print clear error messages and exit gracefully.
- **No prekey:** If Matteo runs before Benny uploads his key, the server returns HTTP 404 and Matteo exits with an instruction to start Benny first.

9.3 Key Verification

We added temporary debug prints during development to confirm that the shared secret computed by Matteo (`DH(matteo_priv, benny_pub)`) and Benny (`DH(benny_priv, matteo_ephemeral)`) were byte-identical. Both produced the same 32-byte value, confirming the correctness of the X25519 key exchange.

10 Conclusion

In this project, we built MetaChat, an end-to-end encrypted messaging system in Go. The system demonstrates how modern secure messaging works at a fundamental level: X25519 key exchange, HKDF key derivation, Double Ratchet for forward secrecy, and AES-256-GCM for authenticated encryption.

The relay server acts as a zero-knowledge intermediary — it stores and forwards ciphertext without ever accessing plaintext. This mirrors the design philosophy of production messengers like Signal.

While the implementation is educational and lacks some production features (TLS, identity verification, replay protection), it provides a clear, working demonstration of the cryptographic principles that protect billions of messages every day.

References

[1] Marlinspike, M. and Perrin, T. "The X3DH Key Agreement Protocol," Signal Foundation, 2016.

[2] Perrin, T. and Marlinspike, M. "The Double Ratchet Algorithm," Signal Foundation, 2016.

[3] Bernstein, D.J. "Curve25519: New Diffie-Hellman Speed Records," PKC 2006.

[4] Dworkin, M. "Recommendation for Block Cipher Modes of Operation: Galois/-Counter Mode," NIST SP 800-38D, 2007.

[5] Krawczyk, H. and Eronen, P. "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, 2010.